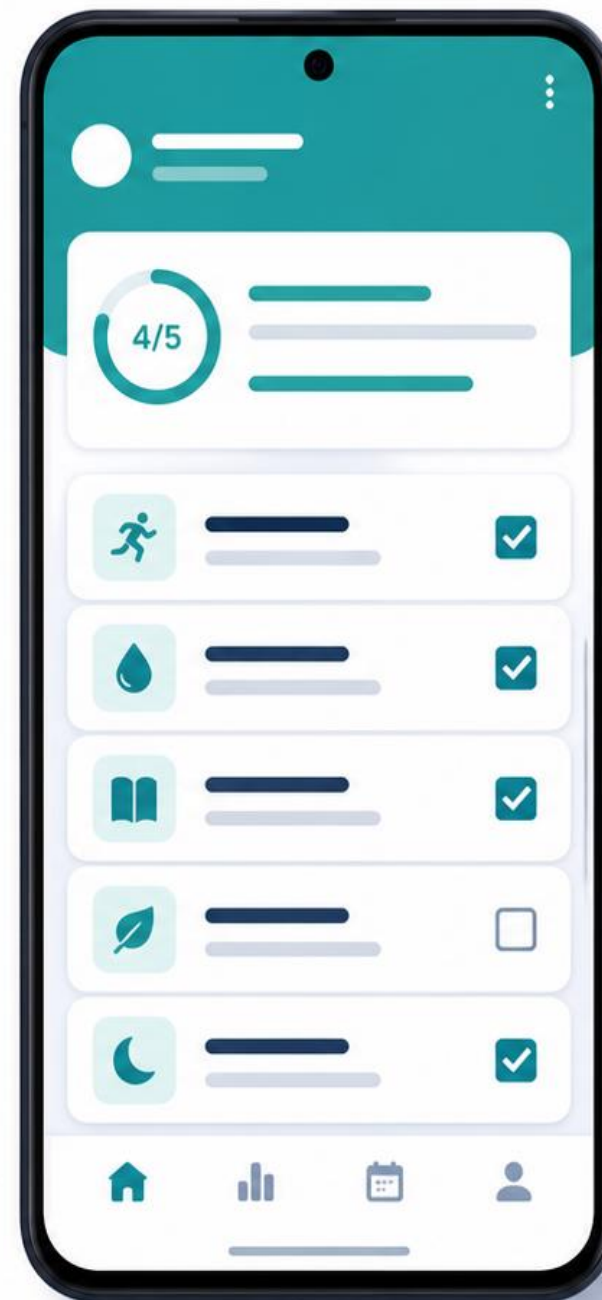


ANDROID · KOTLIN · JETPACK COMPOSE

Mi Ruta de Hábitos

Presentación paso a paso para iniciar con interfaces declarativas en Android Studio.

Proyecto guiado: lista de hábitos, estado, tarjetas y eventos.



Mapa de aprendizaje

Lo que se construye y lo que se aprende con cada bloque de código.

1

Modelo

data class Habito

2

Entrada

MainActivity + setContent

3

Estado

remember + mutableStateOf

4

Interfaz

MaterialTheme, Surface, Column

5

Interacción

TextField, Button, Checkbox

6

Lista

LazyColumn + items

Idea central

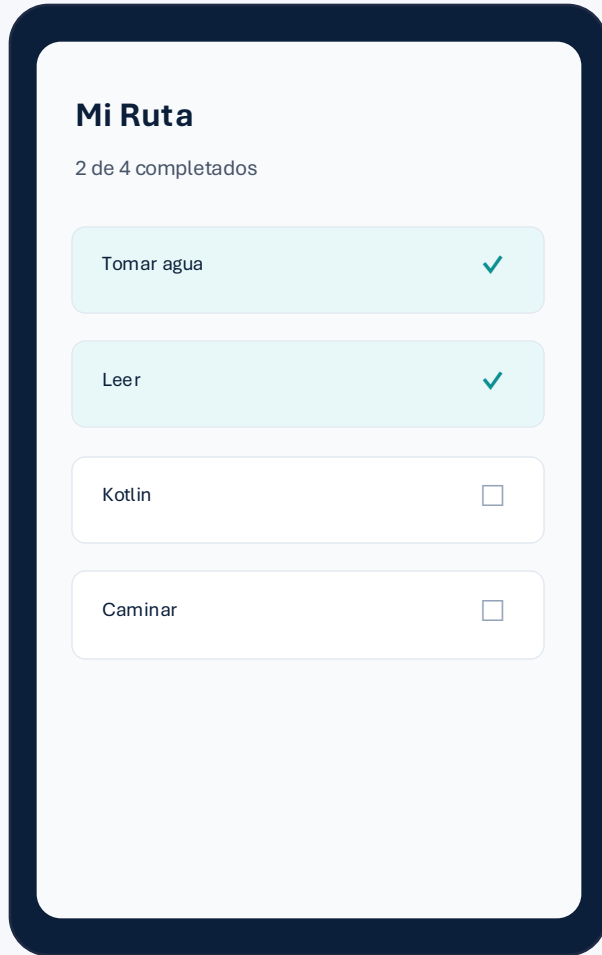
Compense permíteme describir la pantalla con una función en Kotlin. Cuando cambie el estado, la pantalla se recompondrá y mostrará los nuevos datos.





¿Qué app construiremos?

Una app funcional para registrar y completar hábitos personales.



Funcionalidades

1. Mostrar hábitos iniciales
2. Agregar un nuevo hábito
3. Marcar hábitos como completados
4. Calcular un resumen de avance
5. Renderizar la lista de forma eficiente

Aprendizajes técnicos

Kotlin básico, funciones composables, estado en memoria, eventos de usuario, listas con LazyColumn y componentes Material 3.

Producto mínimo viable

No usa base de datos todavía: el objetivo es entender primero cómo fluye la información dentro de la interfaz.

Paso 1 · Imports: traer las herramientas

Los imports indican qué piezas de Android, Compose y Material se usarán.

```
1 import android.os.Bundle
2 import androidx.activity.ComponentActivity
3 import androidx.activity.compose.setContent
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.foundation.lazy.LazyColumn
6 import androidx.compose.foundation.lazy.items
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.*
9 import androidx.compose.ui.Alignment
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.text.font.FontWeight
12 import androidx.compose.ui.unit.dp
```

Lectura didáctica

Cada import habilita una familia de elementos:

- layout: estructura visual
- lazy: listas eficientes
- material3: botones, tarjetas, checkbox
- runtime: estado y recomposición
- ui: alineación, modificadores y unidades

¿Qué Podemos hacer para ver como funciona el autocompletado?

*borrar temporalmente un import y observar el error.

Paso 2 · Modelo de datos

Antes de dibujar la interfaz, definimos qué es un hábito para la app.

```
1 data class Habito(  
2     val nombre: String,  
3     var completado: Boolean = false  
4 )
```

```
1 val ejemplo = Habito("Caminar")  
2 // nombre = "Caminar"  
3 // completado = false
```

¿Qué representa?

Un hábito tiene un nombre y un estado. Por defecto, todo hábito inicia como pendiente porque completado = false.

Kotlin clave

data class crea automáticamente funciones útiles como copy(), equals(), toString() y destructuring. En este proyecto se usa copy() para cambiar el estado sin modificar directamente la lista original.

Paso 3 · MainActivity: punto de entrada

Android abre la Activity y Compose dibuja la pantalla dentro de setContent.

```
1 class MainActivity : ComponentActivity() {  
2     override fun onCreate(savedInstanceState:  
Bundle?) {  
3         super.onCreate(savedInstanceState)  
4         setContent {  
5             MiRutaHabitosApp()  
6         }  
7     }  
8 }
```

Traducción simple

MainActivity es la puerta principal de la app. onCreate se ejecuta cuando la pantalla se crea. setContent indica que la interfaz será construida con Compose.



Paso 4 • Función composable principal

Una función @Composable describe una parte de la interfaz.

```
1 @Composable
2 fun MiRutaHabitosApp() {
3     // Aquí vive el estado y la
   estructura visual
4 }
```

```
1 @Composable
2 fun Saludo(nombre: String) {
3     Text("Hola, $nombre")
4 }
```

@Composable

Esta anotación le dice a Compose que la función puede emitir elementos visuales en pantalla: textos, columnas, botones, tarjetas y listas.

Regla práctica

Piensa en cada composable como una pieza reutilizable de UI. Primero construiremos la pantalla completa y después separaremos la tarjeta HabitoCard.

Ejemplo mínimo: una función composable puede recibir datos y mostrarlos.

Paso 5 · Estado: lo que la pantalla recuerda

El estado guarda datos que cambian por interacción del usuario.

```
1 var nuevoHabitto by remember { mutableStateOf("") }
2
3 var listaHabitto by remember {
4     mutableStateOf(
5         listOf(
6             Habitto("Tomar agua"),
7             Habitto("Leer 10 minutos"),
8             Habitto("Estudiar Kotlin")
9         )
10    )
11 }
```

nuevoHabitto

Almacena el texto escrito por el usuario en el campo de entrada.

listaHabitto

Almacena la lista visible en pantalla. Inicia con tres hábitos de ejemplo.

remember + mutableStateOf

Cuando este valor cambia, Compose vuelve a ejecutar las partes necesarias de la UI para reflejar el nuevo estado.

Recuerda: UI = función del estado.

Paso 6 · Contador de hábitos completados

count recorre la lista y cuenta los elementos que cumplen una condición.

```
1 val completados = listaHabitos.count {  
  it.completado }
```

```
1 listOf(  
2     Habito("Tomar agua", true),  
3     Habito("Leer", false),  
4     Habito("Kotlin", true)  
5 ).count { it.completado }  
6  
7 // Resultado: 2
```

¿Qué hace it?

Dentro de la lambda, it representa cada hábito de la lista. Si it.completado es true, se cuenta.

Puente con lógica

¿qué vemos aquí?

funcional básica en Kotlin: listas, lambdas y condiciones booleanas.

Paso 7 • Estructura visual de la pantalla

Compose organiza la UI como un árbol de funciones.

```
1 MaterialTheme {  
2     Surface(modifier = Modifier.fillMaxSize()) {  
3         Column(  
4             modifier = Modifier  
5                 .padding(20.dp)  
6                 .fillMaxSize()  
7         ) {  
8             // Contenido de la pantalla  
9         }  
10    }  
11 }
```

MaterialTheme

Tema visual

Surface

Contenedor base

Column

Orden vertical

Contenido

Textos, input, botón, lista

Modifier

Los modificadores son instrucciones visuales: tamaño, margen, padding, alineación, color, etc. Se pueden encadenar.

Paso 8 · Encabezado y resumen visual

Text y Spacer construyen la parte superior de la pantalla.

```
1 Text(  
2     text = "Mi Ruta de Hábitos",  
3     style = MaterialTheme.typography.headlineMedium,  
4     fontWeight = FontWeight.Bold  
5 )  
6  
7 Spacer(modifier = Modifier.height(8.dp))  
8  
9 Text(  
10    text = "Hábitos completados: $completados de  
    ${listaHabitos.size}",  
11    style = MaterialTheme.typography.bodyLarge  
12 )
```

Text

Muestra texto en pantalla. Aquí se usa para el título y el resumen.

Spacer

Agrega espacio entre elementos. Es una forma clara de separar visualmente bloques.

String template

Kotlin permite insertar variables dentro del texto usando \$variable o \${expresión}.

Paso 9 · Captura de texto con OutlinedTextField

El campo refleja el estado y actualiza el estado al escribir.

```
1 OutlinedTextField(  
2     value = nuevoHabito,  
3     onChange = { nuevoHabito = it },  
4     label = { Text("Nuevo hábito") },  
5     modifier = Modifier.fillMaxWidth()  
6 )
```

value

Es lo que el campo muestra actualmente.

onChange

Se ejecuta cada vez que el usuario escribe. El nuevo texto llega como it.

Si el usuario escribe “Caminar”, nuevoHabito cambia de "" a "Caminar" y el campo muestra el nuevo valor.

Nuevo hábito: Caminar

Paso 10 · Agregar hábitos con un botón

El botón valida, agrega un elemento y limpia el campo.

```
1 Button(  
2     onClick = {  
3         if (nuevoHabito.isNotBlank()) {  
4             listaHabitos = listaHabitos + Habito(nuevoHabito)  
5             nuevoHabito = ""  
6         }  
7     },  
8     modifier = Modifier.fillMaxWidth()  
9 ) {  
10     Text("Agregar hábito")  
11 }
```

Validación

`isNotBlank()` evita agregar hábitos vacíos o solo con espacios.

Actualizar lista

`listaHabitos + Habito(nuevoHabito)` crea una nueva lista con el hábito agregado.

Limpiar campo

`nuevoHabito = ""` borra el texto del input después de agregar.

Secuencia: escribir → presionar botón → cambiar estado → recomponer UI.

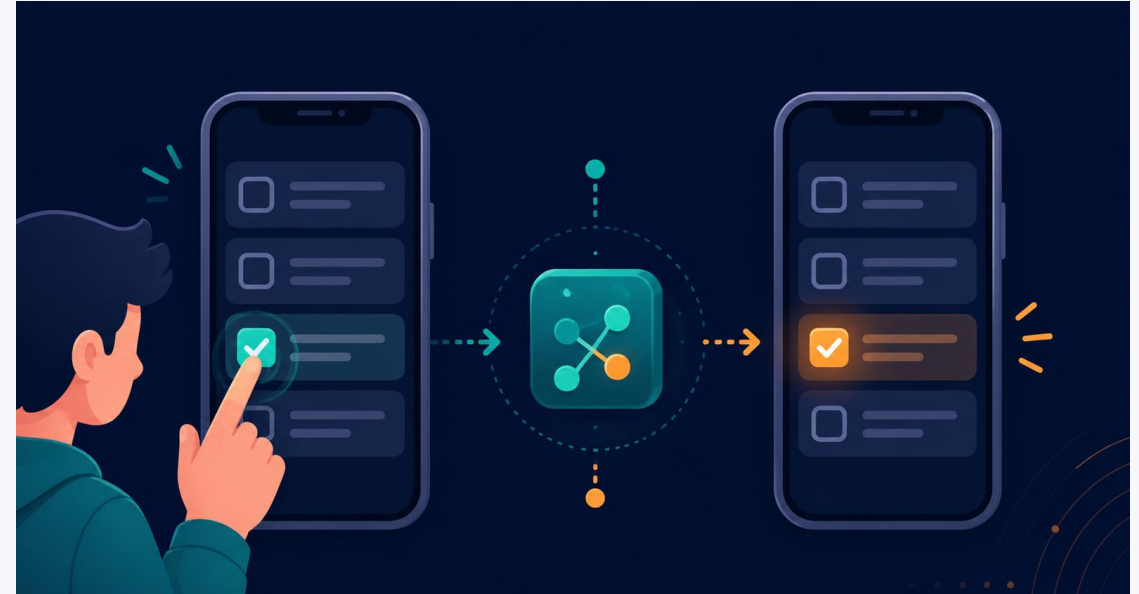
Paso 11 • Mostrar listas con LazyColumn

LazyColumn es ideal para listas verticales con muchos elementos.

```
1 LazyColumn {  
2     items(listaHabitos) { habito ->  
3         HabitoCard(  
4             habito = habito,  
5             onCambioEstado = { ... }  
6         )  
7     }  
8 }
```

items(listaHabitos)

Por cada elemento de la lista, Compose llama a HabitoCard y le entrega un hábito específico.



Ventaja

Permite conectar estructuras de datos con pantalla: si la lista crece, la interfaz crece automáticamente.

Paso 12 · HabitoCard: componente reutilizable

Una tarjeta recibe datos y una acción; así se separa la UI en piezas pequeñas.

```
1 @Composable
2 fun HabitoCard(
3     habito: Habito,
4     onCambioEstado: () -> Unit
5 ) {
6     Card(...) {
7         Row(...) {
8             Column {
9                 Text(text = habito.nombre)
10                Text(text = if (habito.completado) "Completado" else
"Pendiente")
11            }
12            Checkbox(...)
13        }
14    }
15 }
```

Parámetro habito

Entrega los datos que la tarjeta debe mostrar: nombre y estado.

Parámetro onCambioEstado

Entrega una acción que la tarjeta puede ejecutar cuando el usuario interactúa con el Checkbox.

¿por qué es una Buena práctica separar components?

Separar componentes hace el código más legible, reutilizable y fácil de probar.

Paso 13 · Diseño interno de la tarjeta

Card, Row y Column combinan estructura visual y jerarquía.

```
1 Card(  
2     modifier = Modifier  
3         .fillMaxWidth()  
4         .padding(vertical = 6.dp),  
5     elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)  
6 ) {  
7     Row(  
8         modifier = Modifier  
9             .padding(16.dp)  
10            .fillMaxWidth(),  
11        verticalAlignment = Alignment.CenterVertically,  
12        horizontalArrangement = Arrangement.SpaceBetween  
13    ) { ... }  
14 }
```

Card

Agrupar visualmente cada hábito como una unidad independiente.

Row

Coloca el texto a la izquierda y el checkbox a la derecha.

Arrangement.SpaceBetween

Empuja los elementos a extremos opuestos del ancho disponible.

Elevation

Genera profundidad visual mediante sombra, útil para distinguir tarjetas.

Paso 14 · Checkbox y cambio de estado

Al marcar un hábito, se ejecuta una función que cambia la lista.

```
1 Checkbox(  
2     checked = habito.completado,  
3     onChange = { onCambioEstado() }  
4 )
```

```
1 listaHabitos = listaHabitos.map {  
2     if (it.nombre == habito.nombre) {  
3         it.copy(completado = !it.completado)  
4     } else {  
5         it  
6     }  
7 }
```

checked

Vincula el valor visual del checkbox con `habito.completado`.

map

Recorre la lista y construye una nueva. Solo modifica el hábito seleccionado.

copy

Crea una copia del hábito con `completado` invertido. Esto evita mutaciones confusas y favorece un flujo de datos más claro.

Flujo completo de interacción

Así viaja la información cuando el usuario agrega o completa un hábito.



Principio clave

En Compose, no se “empuja” manualmente cada cambio visual. Se actualiza el estado y la UI se vuelve a calcular con los nuevos datos.

Frase para recordar

La pantalla es una fotografía del estado actual.

Errores frecuentes y cómo tratarlos (no morir en el intento)

Anticipar problemas ayuda a depurar.

Problema	Revisión sugerida
No aparece el hábito nuevo	Revisar que listaHabitos se reasigne: <code>listaHabitos = listaHabitos + ...</code>
El botón agrega vacíos	Usar <code>isNotBlank()</code> antes de agregar.
El checkbox no cambia	Confirmar que <code>onCambioEstado</code> actualiza <code>listaHabitos</code> con <code>map/copy</code> .
Texto muy pegado	Ajustar <code>Spacer</code> , <code>padding</code> y <code>verticalArrangement</code> .
Error de import	Usar <code>Alt+Enter</code> o revisar que el componente exista en <code>material3/foundation</code> .

Ahora, ya quedó claro ... ¿qué haremos ?

NA

Evitar hábitos repetidos

Usar `any { it.nombre == nuevoHbito }`

B

Eliminar hábitos

Agregar botón y usar filter

C

Categorías

Modificar data class con categoria

D

Resumen porcentual

Calcular `completados / total * 100`

conceptos que deben quedar claros – con qué nos quedamos

Lo esencial antes de pasar a navegación, ViewModel y persistencia.

1. Kotlin modela los datos

Habito define qué información se usa en la app.

2. Compose dibuja desde funciones

Las pantallas se describen con @Composable.

3. El estado controla la UI

remember + mutableStateOf guardan información cambiante.

4. Los eventos modifican estado

onClick, onValueChange y onCheckedChange responden al usuario.

5. Las listas se renderizan con LazyColumn

items transforma datos en componentes visuales.

6. Los componentes se reutilizan

HabitoCard separa responsabilidad y mejora legibilidad.

¿cuál es la tarea del miércoles? : convertir el estado local en ViewModel y persistir los hábitos con Room y/o DataStore (¿cuál y por qué?)

Fuentes de referencia

¿ Dónde encuentro todo esto ? En la documentación oficial

1

State and Jetpack Compose

developer.android.com/develop/ui/compose/state

2

Lists and grids in Compose

developer.android.com/develop/ui/compose/lists

3

Material Design 3 in Compose

developer.android.com/develop/ui/compose/designsystems/material3

4

Get started with Jetpack Compose

developer.android.com/develop/ui/compose/documentation